

Implement Client Script & UI Policy (Incident)

Project Name: Implement Client Script & UI Policy (Incident)
Team Code: SWTID-2026-7578
Date: October 3, 2026

PROBLEM STATEMENT

Incident records often require consistent and accurate data entry to ensure effective triage, routing, and resolution. However, relying solely on user awareness and manual checks can lead to incomplete, inconsistent, or incorrect data being submitted. This can impact reporting accuracy, SLA compliance, and overall service quality. To address this challenge, there is a need to enforce conditional field behavior and validation directly at the user interface level. Implementing standardized UI Policies and Client Scripts helps control field visibility, mandatory status, and save-time validation, ensuring that Incident records meet defined business rules before they are submitted.

OBJECTIVE

The objective of the **Implement Client Script & UI Policy (Incident)** project is to demonstrate how ServiceNow client-side controls can be used to enforce data integrity on Incident records. This implementation showcases how UI Policies and Client Scripts can dynamically make fields mandatory, auto-populate values, control field behavior, and prevent record submission when required conditions are not met. The solution ensures that Incidents are created with complete and valid information, improving data quality and consistency across the platform.

SKILLS

Incident Management

UI Policy

UI Policy Actions

Client Scripts

Form Validation

IMPLEMENTATION TASKS

Task 1: Create UI Policy on Incident

Activity 1: Create UI Policy

1. Log in to the ServiceNow instance with administrative or configuration access.
2. Navigate to: **System UI → UI Policies**
3. Click **New** to create a new UI Policy record.
4. Fill in the following details:
 - **Name:** High Impact Control (clearly represents the purpose of applying additional controls when an Incident has high impact).
 - **Table:** Incident
 - **Active:** **true**
5. Configure the **Conditions** section:
 - **Field:** Impact
 - **Operator:** is
 - **Value:** 1 – High
6. This condition ensures the UI Policy is triggered only when the Incident impact is set to High.
7. And UI policy action Assignment group → Check **Mandatory**.
8. Set **Reverse if false** to **true** (ensures that when condition is not met, field changes applied by the policy are automatically reverted).
9. Review the configuration to ensure accuracy and click **Submit** to save.

Task 2: Create UI Policy Action – Urgency

Activity 1: Configure the Urgency Field

1. Open the **High Impact Control** UI Policy created earlier. Scroll down to the UI Policy Actions related list.
2. Click **New** to create a UI Policy Action.
3. Fill in the Action Details:
 - **Field name:** Urgency
 - **Read-only:** **true** (makes Urgency read-only whenever Impact = High).
 - **Visible:** Leave as is (field remains visible).
4. Click **Submit** to save the action.
5. *Optional Verification:* Open an Incident, set Impact to High, and confirm Urgency becomes read-only. Change Impact to confirm it reverts.

Task 3: Create onChange Client Script

Activity 1: Create Client Script

1. Navigate to: **System UI → Client Scripts** and click **New**.
2. Fill in the details:
 - **Name:** Auto set urgency for high impact
 - **Table:** Incident
 - **Type:** onChange
 - **Field name:** Impact
 - **Active:** **true**
3. Paste the script in the script editor:

```
function onChange(control, oldValue, newValue, isLoading) {  
    if (isLoading || newValue == '') {  
        return;  
    }  
    if (newValue == '1') {  
        g_form.setValue('urgency', '1');  
        g_form.addInfoMessage('Urgency set to High for High impact incident.');    }  
}
```

4. Click **Submit** to save and verify behavior on an Incident record.

Task 4: Create onSubmit Client Script (Save Validation)

Activity 1: Create Client Script

1. Navigate to: **System UI → Client Scripts** and click **New**.
2. Fill in the details:
 - **Name:** Prevent save if Assigned To missing
 - **Table:** Incident
 - **Type:** onSubmit
 - **Active:** **true**
3. Paste the script into the script editor section:

```
function onSubmit() {  
    if (g_form.getValue('impact') == '1' && g_form.getValue('assigned_to') == '') {  
        g_form.showErrorMessage(  
            'assigned_to',  
            'Assigned To is mandatory for High impact incidents.'  
        );  
        return false;  
    }  
    return true;  
}
```

4. Click **Submit** to save.

Task 5: Create onCellEdit Client Script

Activity 1: Create Client Script

1. Navigate to: **System UI → Client Scripts** and click **New**.

2. Fill in the details:

- **Name:** Prevent state change via list edit
- **Table:** Incident
- **Type:** onCellEdit
- **Field name:** State
- **Active:** **true**

3. In the Script field, paste the following code:

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {  
    alert('State cannot be updated using list editing. Please open the Incident.');
```

```
    callback(false);  
}
```

4. Click **Submit**.

TASK 6: TESTING THE CONFIGURATION

Activity 1: Test Mandatory Enforcement

Navigate to Incident → Create New. Set Impact to High and leave Assigned To empty. Attempt to submit.

Result: Incident is not saved; error message appears indicating Assigned To cannot be empty.

Activity 2: Test Successful Save

Fill Assigned To field and submit.

Result: Incident saves successfully. Urgency auto-sets to High, field becomes read-only, and state restrictions operate as expected.

Activity 3: Reverse Condition Test

Change Impact from High to Medium on an existing Incident.

Result: Assigned To is no longer mandatory and Urgency becomes editable again.

Activity 4: Test List Edit Blocking

Go to Incident → All. Attempt to double-click and edit State directly in list view.

Result: Alert message appears and blocks direct list edit.

Activity 5: Test Form-Based Update

Open Incident record, update State on form, and click Update.

Result: State change saves successfully via the form.

CONCLUSION

The **Implement Client Script & UI Policy (Incident)** micro project effectively demonstrates how UI Policies and Client Scripts can work together to enforce dynamic field behavior, automate updates, and prevent incorrect data submission on Incident forms. By using **onChange**, **onSubmit**, and **onCellEdit** scripts alongside UI Policy actions, the project ensures fields like **Assigned To** and **Urgency** respond appropriately to changes in **Impact**, maintaining clean and consistent data. This solution not only enforces mandatory fields and controls user interactions but also remains lightweight, efficient, and easy to implement, making it ideal for short-duration micro projects while showcasing practical best practices for improving form usability and data integrity in ServiceNow.